

ML Assignment 2

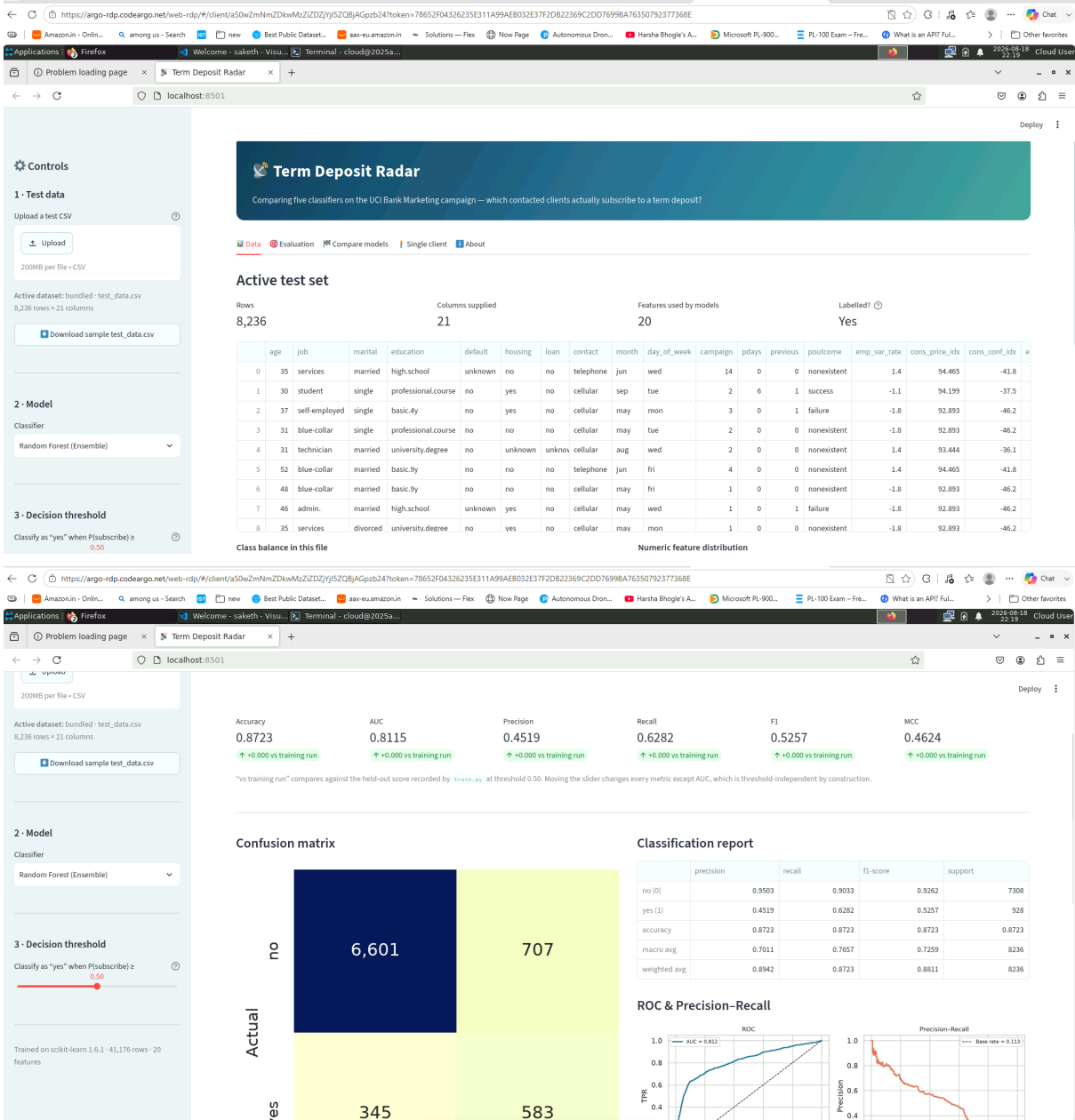
P.Ram Saketh

2025AC05807

Github URL: <https://github.com/sakethp7/sakethp7-ml-assignment2/>

Streamlit URL: <https://sakethp7-ml-assignment2.streamlit.app/>

Screenshot:



Readme content:

Bank Marketing - Term Deposit Prediction

Machine Learning Assignment 2, BITS Pilani WILP M.Tech (AIML/DSE).
Five classification models trained on the UCI Bank Marketing dataset, with a Streamlit app to evaluate them on test data.

- Live app: <https://sakethp7-ml-assignment2.streamlit.app/>
- GitHub repository: <https://github.com/sakethp7/sakethp7-ml-assignment2>

a. Problem Statement

A Portuguese bank runs phone campaigns to sell term deposits. Only about 11% of the contacted clients subscribe, so most calls are wasted effort. The task is binary classification: using what the bank knows about a client before the call (demographics, previous campaign history, economic indicators), predict whether the client will subscribe (`y` = yes/no). Because the classes are imbalanced, accuracy alone is misleading (always predicting "no" already gives ~89%), so models are compared mainly on AUC, F1 and MCC.

b. Dataset Description

```
|||
|---|---|
| Name | Bank Marketing |
| Source | UCI Machine Learning Repository, id 222
| (https://archive.ics.uci.edu/dataset/222/bank+marketing) |
| File used | bank-additional-full.csv |
| Instances | 41,188 raw, 41,176 after removing 12 duplicate rows (minimum required: 500) |
| Input features | 20 (minimum required: 12) |
| Target | `y` - subscribed to a term deposit (yes/no) |
| Class balance | 11.3% yes, 88.7% no |
```

Feature groups:

- Client: age, job, marital, education, default, housing, loan
- Current campaign: contact, month, day_of_week, campaign
- Previous campaigns: pdays, previous, poutcome
- Economic indicators: emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed

Preprocessing decisions:

1. `duration` (call length) is dropped. It is only known after the call ends, so it leaks the target. The UCI documentation itself says it should be discarded for a realistic model.
2. `pdays = 999` means "never contacted before", not 999 days. It is replaced by a new binary column `was\_contacted\_before` and the 999 values are set to 0.

All preprocessing (median/most-frequent imputation, StandardScaler, one-hot encoding) is inside a scikit-learn Pipeline per model, fitted on the training split only. Split: stratified 80/20, random_state = 42.

c. GitHub Repository Link

<https://github.com/sakethp7/sakethp7-ml-assignment2>

...

```
|— app.py          # Streamlit app (inference only)
|— train.py        # training script
|— requirements.txt # pinned dependencies
|— README.md
|— test_data.csv   # held-out 20% test split (8,236 rows)
|— model/          # saved pipelines for all 5 models + metadata
|— reports/        # plots from the training run
|—
```

d. Models Used

```
| # | Model | Main settings |
|---|---|---|
| 1 | Logistic Regression | C=1.0, max_iter=2000, class_weight=balanced |
| 2 | Decision Tree | max_depth=8, min_samples_leaf=25, class_weight=balanced |
| 3 | kNN | k=25, distance weights |
| 4 | Naive Bayes | GaussianNB (Gaussian chosen because scaled features have negative values, which MultinomialNB does not accept) |
| 5 | Random Forest (Ensemble) | 400 trees, max_depth=16, class_weight=balanced_subsample |
```

Comparison Table

Computed on the held-out test set (test_data.csv, 8,236 rows) at threshold 0.50. These are the same numbers the Streamlit app shows for the bundled test file.

```
| ML Model Name | Accuracy | AUC | Precision | Recall | F1 | MCC |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.8304 | 0.8001 | 0.3593 | 0.6455 | 0.4617 | 0.3930 |
| Decision Tree | 0.8230 | 0.7990 | 0.3499 | 0.6659 | 0.4588 | 0.3920 |
| kNN | 0.8969 | 0.7633 | 0.5961 | 0.2640 | 0.3659 | 0.3504 |
| Naive Bayes | 0.8329 | 0.7761 | 0.3476 | 0.5506 | 0.4262 | 0.3463 |
| Random Forest (Ensemble) | 0.8723 | 0.8115 | 0.4519 | 0.6282 | 0.5257 | 0.4624 |
```

Observations

```
| ML Model Name | Observation about model performance |
|---|---|
```

| Logistic Regression | Good baseline. With balanced class weights it catches most subscribers (recall 0.65) but at low precision (0.36), so it flags many false positives. AUC 0.80 is close to the best models despite being a simple linear model. |

| Decision Tree | Very similar numbers to logistic regression once depth is limited (depth 8). Without the depth cap it overfits badly. Its probability estimates are coarse, so its AUC stays slightly below the forest. |

| kNN | Highest accuracy (0.897) but the worst AUC (0.763) and very low recall (0.26). It has no class weighting, so the 89% majority class dominates the neighbourhoods and it misses most subscribers. Also the slowest at prediction time. Good example of why accuracy is misleading on imbalanced data. |

| Naive Bayes | Fastest to train and predict, but its independence assumption does not hold here (the economic indicators are strongly correlated), so it ends up with the lowest MCC (0.346). Middle-of-the-road on everything else. |

| Random Forest (Ensemble) | Best AUC (0.812), best F1 (0.526) and best MCC (0.462). Averaging many decorrelated trees fixes the single tree's variance problem. The tradeoff is a large model file and less interpretability. |

| Overall Winner for your dataset? | Random Forest. It leads on the three metrics that matter under class imbalance (AUC, F1, MCC). kNN's higher accuracy comes from mostly predicting "no", which is useless for the bank. |

Streamlit App

Required features:

- CSV upload of test data (sidebar), with the bundled test_data.csv as default
- Model selection dropdown for the 5 models
- Display of all six evaluation metrics
- Confusion matrix and classification report

Extra: a decision threshold slider, a tab comparing all five models at once with ROC curves, and a form to score a single client. Metrics in the app are computed live on whatever file is uploaded; with the bundled test_data.csv at threshold 0.50 they match the table above.

How to Run

Train (downloads the dataset from UCI automatically):

```
...  
pip install -r requirements.txt  
python train.py  
...
```

This writes `model/`, `reports/`, `test\_data.csv` and regenerates `requirements.txt` pinned to the installed library versions. Keeping scikit-learn pinned matters: loading the saved pipelines under a different version can fail.

Run the app locally:

```
```\nstreamlit run app.py\n```
```

Deployment: push the repo to GitHub, then on [share.streamlit.io](https://share.streamlit.io) create a new app pointing at `app.py`. In advanced settings choose Python 3.11 to match the pinned package versions.

## ## Reference

Moro, S., Rita, P., & Cortez, P. (2014). Bank Marketing [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5K306>